

1. Konsep Utama: Offloading GTP-U ke FPGA

Sistem ini dirancang untuk memisahkan beban pemrosesan paket (packet processing) dari CPU Host (ThinkPad X230) ke perangkat keras khusus (FPGA Genesys Virtex-5). FPGA difungsikan sebagai **GTP DPI Engine dan UDP Telemetry**. Dengan memindahkan *parsing* GTP-U ke FPGA, Host dapat berfokus pada fungsi *Control Plane* atau aplikasi visualisasi tanpa terbebani oleh tingginya *throughput User Plane*.

2. Tiga Metode Ingesti Data (Host ke FPGA)

Dokumen ini menawarkan tiga pendekatan arsitektur untuk mengirimkan trafik dari antarmuka radio/host ke FPGA:

- Metode 1: Host Re-encapsulate (Direkomendasikan untuk Kinerja Tinggi)
 - **Mekanisme:** Paket GTP asli dari *User Equipment* (UE) ditangkap oleh mesin pemroses host yang memanfaatkan *kernel bypass* (seperti DPDK atau PF_RING).
 - Host kemudian membungkus ulang (re-encapsulate) RAW GTP tersebut ke dalam header UDP baru dengan *destination port* 9000 tanpa memodifikasi *payload* aslinya.
 - **Alasan:** Ini memungkinkan FPGA untuk dengan mudah mengidentifikasi paket yang perlu diproses hanya dengan melihat port UDP 9000, membuang header terluar (Ethernet/IP/UDP), dan langsung melakukan *parsing* pada GTP Payload.
- **Metode 2:** Zero Copy (Kinerja Ultra-Tinggi & Latensi Rendah)
 - **Mekanisme:** Sangat bergantung pada *User Space Networking* (DPDK, PF_RING ZC, AF_XDP) untuk melewati *kernel network stack*.
 - Paket ditransfer langsung dari *Ring Buffer* NIC di Host ke NIC FPGA **tanpa proses memcpy (memory copy) per paket**.

Kinerja: Menghasilkan latensi perangkat keras yang sangat rendah, yaitu $< 10 \text{ us}$. Ini sangat krusial untuk implementasi UPF (User Plane Function) pada 5G SA yang menuntut *Ultra-Reliable Low Latency Communication* (URLLC).

- Metode 3: TAP/TUN Interface (Prototyping)
 - **Mekanisme:** Pendekatan paling sederhana menggunakan fitur *mirroring* bawaan Linux. Menggunakan iptables dengan tabel mangle dan target TEE untuk menduplikasi paket UDP port 2152 (port standar GTP-U 3GPP).
 - Paket asli diteruskan (*routing normal*), sementara salinannya dikirim ke IP FPGA (192.168.1.20). Pendekatan ini sangat cocok untuk tahap PoC (Proof of Concept) di lingkungan riset.

3. Pipeline Pemrosesan di Dalam FPGA (Penerimaan)

Setelah paket masuk melalui antarmuka RJ45, FPGA melakukan pemrosesan *pipeline* berbasis *hardware block*:

1. **MAC & IP Layer:** Modul XC5V TEMAC memproses lapisan Ethernet, dilanjutkan oleh IPv4 Parser yang mengekstrak *payload* UDP.
2. **UDP Filtering:** Komparator perangkat keras memeriksa apakah *destination port* adalah 9000. Jika cocok (MATCH), *payload* diekstrak sebagai RAW GTP PACKET.
3. **GTP Parsing & State Machine:** Parser GTP mengekstrak elemen standar 3GPP v1: *Flags* (0x30), *Message Type* (0xFF / T-PDU), *Length*, dan *TEID* (Tunnel Endpoint Identifier). Arsitektur logis ini juga mencakup modul lanjutan seperti *TEID Lookup*, *Flow Counter*, *Session State Machine*, dan *Pattern Match* untuk DPI.

4. Estimasi Sizing & Kapasitas (Konteks 2G/GPRS)

Sistem diuji dengan skenario lalu lintas 2G/GPRS yang realistis:

- Jika sistem menerima beban lalu lintas 20,000 packet/s dengan ukuran rata-rata paket 250 byte
- Data rate asli (RAW) dihitung sebagai: $20,000 \times 250 = 5,000,000 \text{ byte/s} = 5 \text{ MB/s}$ atau 40 Mbps
- Dengan tambahan *overhead* enkapsulasi (Ethernet 14 byte + IP 20 byte + UDP 8 byte = 42 byte), total estimasi *data rate* menjadi 45 - 50 Mbps.
- Karena koneksi menggunakan *Gigabit Ethernet* (1 Gbps), utilitas *link* hanya sekitar = 5%, menunjukkan efisiensi tinggi dan ruang yang sangat luas untuk skalabilitas ke trafik 3G/4G/5G.

5. Ekspor Telemetry (Model Komunikasi FPGA ke Host)

Setelah FPGA mengekstrak *insight* dari paket GTP, ia bertindak sebagai *Telemetry Producer* dan mengirimkan data kembali ke Host via UDP (Port 9000).

- **Alasan penggunaan UDP:** Dipilih karena *low latency*, bersifat *non-blocking*, dan tidak memerlukan *handshake*, menjadikannya ideal untuk pengiriman statistik periodik berkecepatan tinggi.
- Format Pesan (Message Types):
 - TYPE 0x01: Global Statistics (periodik tiap detik, misal total IMSI, *uplink pps*).
 - TYPE 0x02: Per TEID Statistics (status IMSI per sesi aktif).

- TYPE 0x03: Event Notification (*real-time*, seperti deteksi pembuatan sesi/Create Session).
- TYPE 0x04: Protocol Distribution (komposisi protokol di dalam GTP-U, penting untuk klasifikasi layanan).
- Beban *feedback* telemetri ini sangat efisien, diperkirakan hanya memakan *bandwidth* = 25 KB/s secara keseluruhan